Python Basics: A Reference Guide

1. Introduction to Python

Python is a high-level, interpreted, general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability with its notable use of significant whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects.

Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including structured (particularly procedural), object-oriented, and functional programming. Python is often described as a 'batteries included' language due to its comprehensive standard library.

2. Built-in Data Types

Python provides a rich set of built-in data types to handle various kinds of data structures efficiently:

• Numeric Types: `int` (arbitrary-precision integers), `float` (double-precision floating point numbers), and `complex` numbers.

• Sequence Types: `str` (immutable strings), `list` (mutable ordered sequences), and `tuple` (immutable ordered sequences).

• Mapping & Set Types: `dict` (mutable key-value hash maps) and `set` / `frozenset` (unordered collections of unique elements).

• Boolean Type: `bool` representing truth values `True` and `False`.

Example of common type usage and immutability:

```
x = 42 name = 'Python' coordinates = (10.0, 20.0) settings = {'debug': True, 'level': 3}
```

3. Control Flow Statements

Control flow in Python determines the order in which statements are executed. Conditional branching is handled via `if`, `elif`, and `else` blocks. Iteration over sequences is performed using `for` loops or condition-controlled `while` loops.

Loops support `break` to exit early, `continue` to skip to the next iteration, and a unique `else` clause that executes only if the loop terminates without encountering a `break` statement.

Control flow example:

```
for num in range(2, 10): if num % 2 == 0: print(f'Found even number: {num}') continue print(f'Found odd number: {num}') else: print('Loop completed successfully!')
```

4. Functions and First-Class Objects

Functions are defined using the `def` keyword. Python supports default argument values, arbitrary positional arguments (`*args`), arbitrary keyword arguments (`**kwargs`), and anonymous inline functions via `lambda` expressions.

In Python, functions are first-class objects. They can be assigned to variables, stored in data structures, passed as arguments to other functions, and returned from functions.

Advanced function definition example:

```
def process_data(multiplier=1, *items, **options): results = [item * multiplier for item in items] if options.get('verbose', False): print(f'Processed {len(results)} items.') return results # Lambda usage square = lambda x: x ** 2
```